

[CSE299(1) - Group#2] Classroom Booking System (CBS) - Priority-Based Faculty Allocation

Tawhid Hasan (2413054042), Mohammad Tasvirul Hasan Riyad (2413692042),
Md. Tanvir Islam (2312491042), Md. Tayasshuk Imam (2411600042)
{tawhid.hasan.241, tasvirul.riyad.241, tanvir.islam2, tayasshuk.imam.241}@northsouth.edu
Summer 2026

I. INTRODUCTION

Every semester, faculty members across university departments compete for a limited pool of classrooms, and the process of matching room requests to available slots is still handled largely through manual coordination spreadsheets, email threads, and phone calls to the registrar’s office. This approach breaks down quickly once multiple instructors request the same room at the same time, offering no principled way to decide who gets priority when a genuine conflict occurs.

Existing scheduling tools on the market are either generic calendar-booking utilities with no concept of institutional hierarchy, or heavyweight ERP modules that are expensive, rigid, and not tailored to the specific rhythms of an academic department. Neither option addresses the core pain point identified by faculty and administrative staff alike: when two people want the same room, there is no transparent, fair, and automatic way to resolve that conflict.

This is where the Classroom Booking System (CBS) comes in: a web-based platform purpose-built to let faculty search for available classrooms, submit one-time or recurring booking requests, and receive automatic, seniority-aware conflict resolution. Rather than leaving disputes to be settled informally, CBS applies a structured academic seniority hierarchy:

Professor → Associate Professor → Assistant Professor → Lecturer

combined with a first-come-first-served tiebreaker, so every faculty member knows exactly why a booking was confirmed or waitlisted.

CBS is designed as a complete lifecycle platform: it manages classroom inventory (rooms, equipment, operating hours, maintenance windows), faculty profiles synced from the institutional HR system, real-time availability search, waitlist promotion, academic calendar synchronization, multi-channel notifications, and a public read-only timetable compliant with Web Content Accessibility Guidelines (WCAG 2.1) [1] for lobby displays and self-study planning. Built on a modern web stack—utilizing React [2], Node.js [3], PostgreSQL [4], and Redis—CBS is engineered to comfortably scale from an initial single-building deployment to a multi-campus system serving thousands of faculty and thousands of bookings per day.

II. BACKGROUND AND RELATED WORK

Room and resource booking is a well-explored problem in general-purpose software tools like Google Calendar’s resource-booking feature, Microsoft Outlook room finder, and various campus ERP scheduling modules that allow a room to be reserved for a time slot. These tools are effective for simple, low-contention booking, but they share a common limitation: when two people want the same resource at the same time, the outcome is typically “first click wins,” with no consideration of institutional role, seniority, or departmental precedence.

In an academic setting, this gap matters. A newly-joined lecturer and a senior professor are not, in practice, treated as equally entitled to a contested lecture hall; departments already operate by informal seniority norms, but no existing booking tool encodes that logic. Generic ERP scheduling modules attempt to solve this with rigid, admin-only allocation, which removes faculty autonomy and turns every conflict into a support ticket for the registrar’s office.

CBS is designed to sit between these two extremes. It preserves the self-service search-and-book experience faculty expect from modern calendar tools, while introducing a first-class, auditable priority engine that automatically resolves conflicts using seniority rank and submission timestamp with a documented “no-bump” policy so that a confirmed booking is never silently revoked once granted. Department Heads retain a manual override for exceptional cases, and every override is logged for accountability, consistent with role-based access control (RBAC) principles [5], where permissions are scoped to a defined role rather than granted broadly. This combination of self-service convenience and structured fairness is the gap CBS is built to fill.

III. PROBLEM IDENTIFICATION

The development of the Classroom Booking System addresses several concrete problems observed in the current manual and semi-manual room-booking process:

- **Inefficient, Manual Conflict Handling:** Faculty currently request rooms through email or spreadsheets, and conflicts are resolved informally and inconsistently by whoever handles the request first. This is slow, error-prone, and offers no documented, repeatable rule for who wins a contested slot.
- **No Structured, Fair Priority System:** There is no existing tool that formalizes academic seniority as a tiebreaker for room conflicts. Decisions are made ad hoc, which can feel arbitrary to faculty and creates friction between departments, especially during high-demand periods like the start of a semester.
- **Lack of Real-Time Visibility and Notifications:** Faculty have no single place to check real-time room availability, track the status of a pending request, or get notified automatically when a waitlisted booking is promoted. This leads to double-bookings, missed slots, and rooms sitting empty because their availability wasn't visible.
- **Fragmented Room & Equipment Data:** Classroom capacity, equipment (projectors, lab benches, computers), and maintenance status are not centrally tracked, so faculty often arrive at a room only to discover it's unsuitable or unexpectedly out of service.

IV. SOLUTION METHODOLOGY

The Classroom Booking System is designed to address each identified issue directly through dedicated modules:

Fig. 1. Landing Page - Institutional SSO Login & Demo Profiles

- **Addressing Inefficient, Manual Conflict Handling:** The Booking Request module allows faculty to submit one-time or recurring room requests directly through the platform. Faculty locate a suitable room through a filtered availability search, ranking matches by capacity, equipment, and proximity to the requested slot, in line with standard information-retrieval ranking and filtering practice [6]. Every submission is timestamped and immediately passed to the Priority-Based

Allocation engine, which runs the conflict check synchronously using PostgreSQL [4] database-level row locking [7], eliminating race conditions when two faculty submit requests for the same room and slot at nearly the same time.

- **Addressing the Lack of a Structured Priority System:** The Priority-Based Allocation module implements the seniority hierarchy directly in the booking engine: Professor (rank 1) through Lecturer (rank 4), with earlier submission timestamp as the tiebreaker for equal-rank requests. Critically, CBS enforces a documented no-bump policy: a confirmed booking is never revoked to make way for a higher-priority late request, so faculty can trust that a confirmation is final. Department Heads are given a scoped, audit-logged override for exceptional cases only.

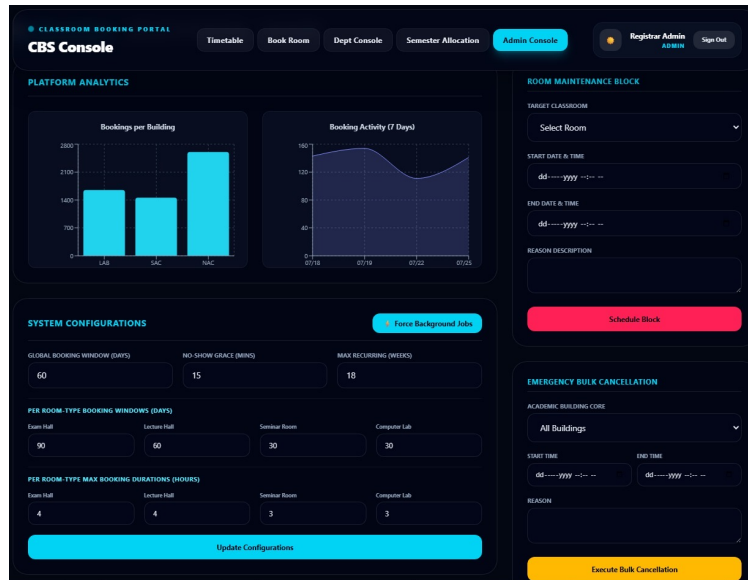


Fig. 2. Admin Console - Analytics, Configurations & Emergency Maintenance

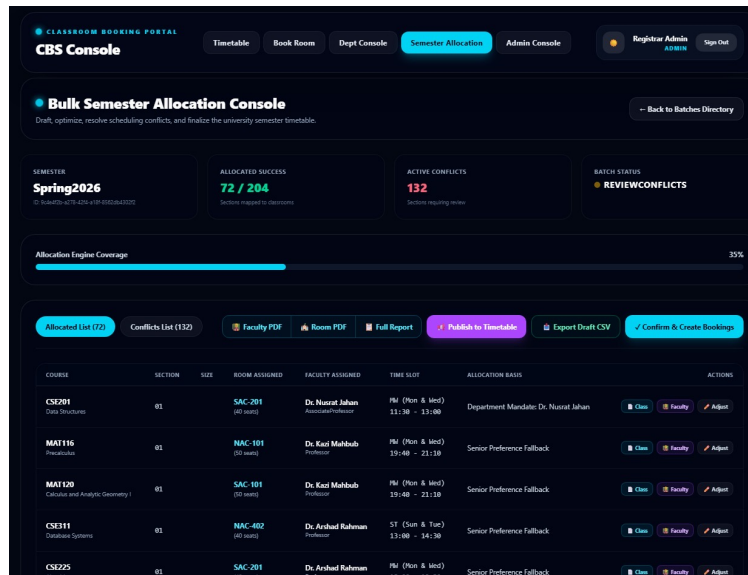


Fig. 3. Semester Allocation Console - Priority Allocation & Conflict Resolution

- **Addressing the Lack of Real-Time Visibility and Notifications:** The Notification module sends transactional email updates via Nodemailer [8], along with SMS and push alerts at every stage of a booking's lifecycle: submission, confirmation, waitlist placement, promotion, and reminders 24 hours and 15 minutes before a slot. The Booking Management module gives faculty a personal, at-a-glance dashboard showing all their bookings, current status, and waitlist position, following established dashboard-design principles [9] for surfacing status information without requiring the user to dig for it.
- **Addressing Fragmented Room & Equipment Data:** The Classroom Inventory module centralizes every bookable space, its equipment profile (projector, smart board, lab benches, computer stations, etc.), operating hours per building, and

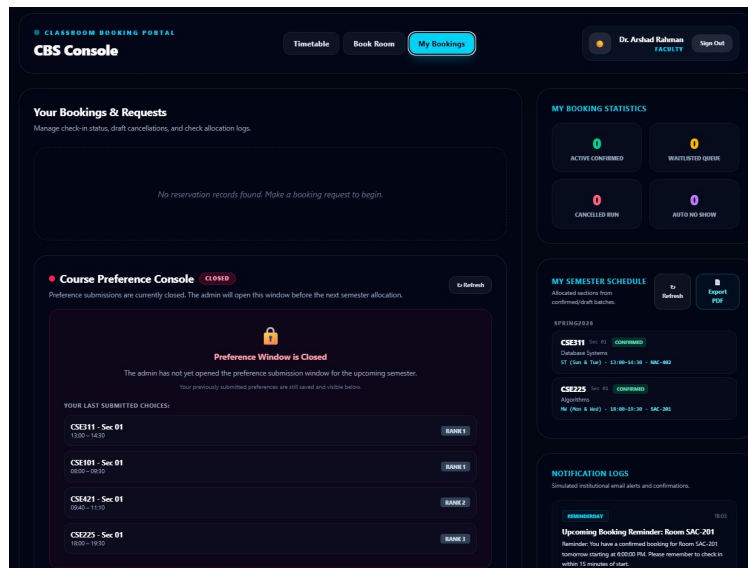


Fig. 4. Faculty Console - Preference Submission, Schedule & Logs

maintenance windows. Admins can bulk-import rooms via CSV, and the system automatically blocks bookings for rooms under maintenance, including notifying any faculty whose existing booking falls inside a newly-scheduled maintenance window. Every authorization change and booking mutation is captured in an append-only action log, an approach consistent with threat-modeling guidance [10] to make privileged actions traceable and reviewable after the fact. Version control workflows and deployment actions are maintained according to branch protection rules [11].

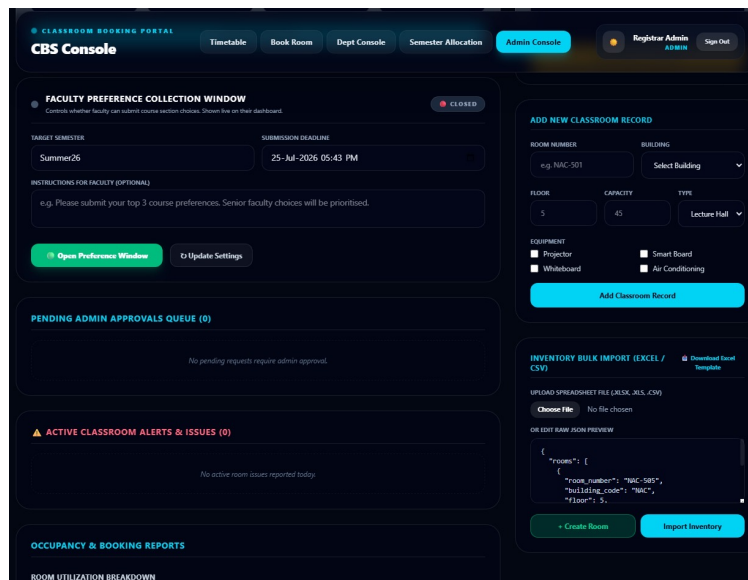


Fig. 5. Inventory Management Console - Preference Window & Bulk Import

V. EXPECTED TIMELINE

The following schedule outlines the 10-week development timeline for CBS, detailing each phase, key activities, and expected deliverables:

VI. CONCLUSION

The Classroom Booking System is designed to replace an inefficient, manual, and inconsistent room-request process with a transparent, self-service platform that faculty can trust. By encoding academic seniority directly into an automated, auditable conflict-resolution engine, and pairing it with real-time search, multi-channel notifications, and centralized inventory management, CBS removes the ambiguity and administrative overhead that currently surrounds classroom booking.

TABLE I
10-WEEK COMPREHENSIVE DEVELOPMENT TIMELINE.

Task List / Module	Period	Status
Foundation & Architecture (Project Setup, DB Design, Auth)	Week 1	Completed
Priority Engine & Booking Rules (Priority Validation, No-Bump)	Week 2	Completed
Dashboards & Advanced Search (Role Dashboards, Search)	Week 3	Completed
Live Notifications & Analytics (SMTP, Admin Charts)	Week 4	In Progress
Concurrency Hardening (Stress Testing, Race-Condition Check)	Week 5	Planned
Recurring Bookings & Calendar Sync (Patterns, Sync)	Weeks 6–7	Planned
Reporting & Public Timetable (Admin Reporting, Lobby View)	Weeks 8–9	Planned
QA & Deployment (E2E Testing, Security Review, Prod)	Week 10	Planned

Our proposed 10-week development plan delivers this system incrementally: starting with foundational data models and authentication, building out search and booking experiences, implementing the priority engine with proper concurrency safety, and finishing with notifications, reporting, and production hardening. Built on a scalable, secure technology stack, CBS is positioned to serve a single department’s needs today while being architected to grow to a multi-campus deployment serving thousands of faculty.

REFERENCES

- [1] World Wide Web Consortium. (2018) Web content accessibility guidelines (wcag) 2.1. [Online]. Available: <https://www.w3.org/TR/WCAG21/>
- [2] React documentation. Meta Open Source. [Online]. Available: <https://react.dev/>
- [3] Node.js documentation. OpenJS Foundation. [Online]. Available: <https://nodejs.org/en/docs>
- [4] Postgresql documentation. PostgreSQL Global Development Group. [Online]. Available: <https://www.postgresql.org/docs/>
- [5] R. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb 1996.
- [6] R. Baeza-Yates and B. Ribeiro-Neto, *Modern Information Retrieval: The Concepts and Technology behind Search*, 2nd ed. Addison-Wesley, 2011.
- [7] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2020.
- [8] Nodemailer - send emails from node.js. [Online]. Available: <https://nodemailer.com/>
- [9] S. Few, *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring*, 2nd ed. Analytics Press, 2013.
- [10] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014.
- [11] Github docs - about branch protection rules. [Online]. Available: <https://docs.github.com/>

INDIVIDUAL CONTRIBUTION

- The tasks were divided across team members based on core system modules to ensure equal division of labor across backend, database, UI, and documentation.

TABLE II
CONTRIBUTION SUMMARY.

Task List	Member Assigned	Comments
Database Schema & Row-Locking Concurrency	Tawhid Hasan	PostgreSQL setup with locking mechanisms
SSO Login & Auth Profile Bypass	Mohammad Tasvirul Hasan Riyad	Next.js Authentication integration
Priority Engine & Seniority Tiebreaker Rules	Md. Tanvir Islam	Rank logic and no-bump validation
Inventory Bulk Importer & Admin Console UI	Md. Tayasshuk Imam	CSV parser and analytics view setup